

Projekt-Validierung

Was geprüft wird, wie geprüft wird und was ein Lauf kostet

Zweck

Die Validierung trennt zwei Dinge, die leicht verwechselt werden: die Prüfung der Konfiguration und die Prüfung des Verhaltens. Die Konfigurationsprüfung ist ein einzelner Aufruf (`rag-project --check`) und beantwortet die Frage, ob Schlüssel, Modell und Adresse zusammenpassen. Die Verhaltensprüfung ist die Pytest-Suite und läuft ohne Netz und ohne echte Schlüssel.

Konfiguration prüfen

Der Schalter `--check` löst das Ziel über `resolve_target()` auf und gibt es aus. Schlägt die Auflösung fehl, erscheint eine sprechende Meldung auf `stderr` und der Rückgabewert ist 1 statt 0.

Ausgabezeile	Herkunft
Provider	Settings.chat_provider bzw. das Präfix der Modellkennung
Modell	ModelInfo.qualified_id und label aus core/models.py
Typ	ModelInfo.kind — flagship, balanced, fast, reasoning oder open_weight
Base-URL	ChatProvider.base_url aus core/config.py
Ziel	ResolvedTarget.summary() — Kennung, Adresse, Temperatur, max_tokens

Wichtig ist, welcher Key geprüft wird. Es wird nur der Schlüssel des aktiven Providers verlangt. Ein reiner OpenRouter-Nutzer braucht keinen OpenAI-Schlüssel und umgekehrt. Erst ein Wechsel des Providers erzwingt die Anwesenheit des anderen Schlüssels — und zwar zur Laufzeit, nicht nur beim Start.

Die Auflösungsreihenfolge des Modells

Drei Quellen können ein Modell bestimmen. Die Reihenfolge ist festgelegt und wird in `resolve_target()` angewandt.

- Das Argument `model`, etwa über `--model` auf der Kommandozeile.
- `CHAT_MODEL` aus den Settings, sofern nicht leer. Ein leerer Wert wirkt wie „nicht gesetzt“.
- Der Standard des Providers aus dem Katalog.

Ein Anbieterpräfix in der Kennung schlägt `CHAT_PROVIDER`. Wer `openai:gpt-4o-mini` schreibt, will OpenAI, auch wenn in der `.env` `openrouter` steht. Ohne Präfix wird zuerst gegen den aktiven Provider aufgelöst; findet sich dort nichts, sucht der Katalog in allen Providern.

Mehrdeutige Kennungen

- Eine Kennung ohne Präfix, die bei mehreren Providern existiert, ist nicht auflösbar. Der Katalog wirft dann einen `UnknownModelError`, statt zu raten.

Testsuite

Die Suite liegt unter `tests/` und umfasst neun Dateien: eine Paket-Initialisierung und acht Prüfdateien. Sie läuft vollständig offline.

Testdatei	Prüfswertpunkt
<code>test_config_and_models.py</code>	Settings-Validierung, <code>SecretStr</code> , Katalog, Provider-Auflösung
<code>test_main.py</code>	Standardausgabe des Einstiegs ohne Argumente
<code>test_main_chatbot.py</code>	Client wird beim Aufbau und beim System-Reset neu gebaut
<code>test_main_cleaning.py</code>	Parser nimmt Wikipedia-Argumente an
<code>test_main_loading.py</code>	geladene Dokumente werden als Textdatei geschrieben
<code>test_paths.py</code>	Pfadauflösung: Projektwurzel, Arbeitsverzeichnis, <code>DOCS_DIR</code>
<code>test_remote_pdf_loader.py</code>	PDF-URL: Download, Metadaten, kein Überschreiben
<code>test_wikipedia_loader.py</code>	Wikipedia-Loader setzt den konfigurierten User-Agent

Was die Suite absichert

Aus den Testdateien lassen sich die Zusicherungen ablesen, die als Vertrag gelten. Sie sind der beste Beleg dafür, welches Verhalten das Projekt garantiert.

- Settings mit einem Provider ohne dessen Schlüssel wirft einen RuntimeError mit dem Namen der Umgebungsvariable in der Meldung.
- Ein SecretStr taucht weder in repr() noch in str() im Klartext auf; describe_settings() gibt nur „set (n chars)“ zurück.
- Eine Temperatur außerhalb von 0.0 bis 2.0 wird abgelehnt; ein aus Leerzeichen bestehendes CHAT_MODEL wird zu None.
- Jeder Provider hat genau ein Standardmodell, und dessen Flag default ist gesetzt.
- Die Kennung samt Präfix lässt sich verlustfrei hin und zurück übersetzen.
- Eine unbekannte Kennung liefert Vorschläge, darunter den nächstliegenden Katalogeintrag.
- Das Präfix schlägt CHAT_PROVIDER; eine doppelte Modell-ID wird im Registry-Konstruktor abgelehnt.
- Ein Modell ohne Temperatur-Unterstützung bekommt den Parameter gar nicht erst gesetzt.
- Der Chatbot baut den Client genau zweimal: beim Aufbau und beim System-Reset, jeweils mit streaming=True.
- Der Ladebefehl schreibt die Inhalte zweier Dokumente getrennt durch eine Leerzeile in die Zielfeile.
- Ein relativer Dateiname wird zuerst im Arbeitsverzeichnis gesucht, dann im konfigurierten Dokumentenordner.
- Eine PDF von einer Adresse wird geladen, mit Quelladresse, Dateiname, Format und Seitengröße in den Metadaten abgelegt und unter docs/ gespeichert.
- Existiert dort bereits eine gleichnamige Datei, bleibt sie unverändert und die neue Fassung landet als scan_1.pdf.
- Eine Adresse, die keine PDF liefert, wird mit der Meldung „keine gueltige PDF-Datei“ abgelehnt.
- Der Wikipedia-Loader liest den User-Agent aus der Umgebung und reicht Sprache, Höchstzahl und Suchbegriff durch.

Befehle

Die Suite wird über uv ausgeführt. Der Abdeckungspfad muss auf src/rag_project zeigen, weil das Paket dort liegt.

- `uv run pytest` — alle Tests
- `uv run pytest tests/test_paths.py -q` — eine einzelne Datei
- `uv run pytest -k pdf -q` — nur Tests, deren Name passt
- `uv run pytest --cov=src/rag_project --cov-report=term-missing -q` — Abdeckung im Terminal
- `uv run pytest --cov=src/rag_project --cov-report=html -q` — HTML-Bericht unter htmlcov/

Konfiguriert ist das in pyproject.toml: testpaths zeigt auf tests, und erkannt werden Dateien nach dem Muster test_*.py. Die Testwerkzeuge — pytest, coverage und pytest-cov — stecken in der Gruppe dev und werden mit `uv sync --group dev` installiert.

Grenzen der Validierung

Die Suite prüft kein Modellverhalten. Sie ruft weder einen echten Anbieter noch ein Embedding-Modell. Antwortqualität, Trefferqualität und Kosten eines Laufs sind damit ausdrücklich nicht abgedeckt; dafür braucht es einen Lauf mit echtem Schlüssel, etwa `rag-project --check` gefolgt von `rag --ask`.